	BTS SIO **Services Informatiques aux Organisations**	
	Option	SLAM
	Session	**2025/26**

Nom et prénom : Costantini Yaacov	**Activité professionnelle N° 8**

Intitulé de l'activité	**Effectuer une mise à jour d'une application web - CMS SIO**
Contexte	Dans le cadre du développement d'un site web de présentation des matières du BTS SIO, une première version fonctionnelle existait mais présentait des limitations structurelles importantes. Il m'a été demandé d'apporter une mise à jour évolutive du projet afin de le faire progresser vers un affichage dynamique.
Objectifs	➢ Modifier le routeur pour qu'il prenne en charge un paramètre dynamique {id} ➢ Refactoriser les routes ➢ Ajouter les requêtes SQL nécessaires dans les modèles ➢ Simplifier l'*IndexController* en remplaçant les fonctions redondantes ➢ Améliorer l'expérience utilisateur en ajoutant une page d'accueil conviviale
Lieu de réalisation	Télétravail

SOLUTIONS ENVISAGEABLES

1. Continuer à créer une fonction et une route par matière, en dupliquant le code existant pour chaque nouvelle ressource. Solution rapide à court terme mais qui aggrave la dette technique, rend le code difficile à maintenir et va à l'encontre du principe DRY. **Écartée pour ces raisons.**
2. Refactoriser le routeur, les routes, les modèles et le contrôleur pour introduire un paramètre dynamique {id}, permettant à une seule route et une seule fonction de gérer toutes les ressources d'un même type. Solution plus exigeante à mettre en œuvre mais qui assainit durablement le code, facilite les évolutions futures et respecte les bonnes pratiques du développement MVC. **Solution retenue.**

DESCRIPTION DE LA SOLUTION RETENUE

Conditions initiales	Le site est fonctionnel mais limité : la page d'accueil expose directement la liste des matières, les pages matière affichent uniquement un titre statique sans contenu dynamique, et l'*IndexController* contient autant de fonctions que de matières (8 fonctions quasi-identiques). Le routeur ne supporte pas les paramètres dynamiques dans les URLs.
Conditions finales	À l'issue de la mise à jour, le site dispose d'une page d'accueil dédiée séparée de la liste des matières. Le routeur supporte les paramètres dynamiques {id}. Les routes sont factorisées. Les modèles *Matiere*, *Theme* et Chapitre exposent les requêtes nécessaires à l'affichage dynamique. L'*IndexController* ne contient plus que trois fonctions dynamiques au lieu des huit redondantes.
Outils utilisés	PHP (architecture MVC, PDO), MySQL, Git (versioning), Navigateur web (tests), VS Code (IDE), Documentation PHP

CONDITIONS DE REALISATION

Matériels	PC Windows 11 Pro, Environnement de développement
Logiciels	VS Code, Wamp, Git, GitHub
Durée	4H
Contraintes	▪ Ne pas modifier l'arborescence du projet ni son architecture générale ▪ Maintenir le site fonctionnel à chaque étape de la modification ▪ Versionner chaque étape significative avec Git

COMPETENCES MISES EN OEUVRE POUR CETTE ACTIVITE PROFESSIONNELLE

Code	Intitulé
B2.1	Conception et développement d'une solution applicative
B2.2	Maintenance corrective ou évolutive d'une solution applicative
B2.3	Gestion des données
1.4	Travail en mode projet
1.5	Mise à disposition des utilisateurs un service informatique
DEROULEMENT DE L'ACTIVITE	

Plan

1. Avant la Maj.
2. La mission
3. Application de la maj. Backend
4. Amélioration de l'interface utilisateur

I. Etat avant la mise à jour

Dans l'état actuel, lorsque l'utilisateur se connecte au site des matières, il arrive sur une page d'accueil contenant la liste des matières et les compétences attestées du BTS SIO comme suit :

Figure 1

Et lorsqu'il clique sur une des matières via les cartes visibles ou via la barre de navigation, il arrive sur une page vide avec uniquement le titre de la matière :

Figure 2

Le code était écrit de façon qu'il fallait répéter l'opération pour chaque matière ce qui est contraire au principe DRY (*Don't Repeat Yourself*) car le code devient redondant.

```
// Bloc 1
0 references | 0 overrides
public function indexSupport($id) {
    $matieres = $this->model->getMatiere();
    $themes = $this->listThemes->getThemeById($id);

    afficherVue('Support et mise à disposition des services informatiques', 'public/support',
        'matieres' => $matieres,
        'themes' => $themes
    ], "/support");
}

// Bloc 2 option SISR
0 references | 0 overrides
public function indexSisr() {
    $matieres = $this->model->getMatiere();
    $themes = $this->listThemes->getTheme();
    afficherVue('Solution Infrastructure Systèmes et Réseaux', 'public/sisr', [
        'matieres' => $matieres,
        'themes' => $themes
    ], "");
}
```

On peut constater la redondance dans le *Controller* principal :
Si on observe bien, on voit que la même fonction se répète pour chaque matière, et il y en a 8, donc, très mauvaise pratique.

Figure 3

II. La mission

Elle consiste à apporter en plusieurs points des modifications et à faire progresser le site dans son évolution. Les tâches demandées et à traiter sont les suivantes :

- Faire en sorte que lorsqu'on clique sur une matière, on puisse avoir accès à une page 'thèmes' de la matière sélectionnée et qui en décline la liste.
- Même principe pour les chapitres : lorsqu'on clique sur un thème, on devrait pouvoir arriver sur une page qui affiche et décline les chapitres du thème sélectionné.
- Accessoirement modifier la page d'accueil afin éviter d'exposer les matières à l'entrée du site : créer un bouton ou un lien permettant d'aller vers une page matière. Cela permet d'une part de clarifier et séparer les parties du site et d'autre part, du point de vue utilisateur, cela peut être désagréable d'avoir le contenu sans une petite accueil conviviale. (Expérience utilisateur - UX)

III. La mise à jour et son exécution

Pour réaliser cette mission, il est nécessaire de faire une modification **structurelle** du code, non pas dans son architecture, qui est relativement propre, mais dans le contenu des modules internes : fonctions, routing / routes, et vues. Je n'ai donc pas à toucher à l'arborescence du projet.

1. Les étapes clés de la modification sont les suivantes :

- Modification du routeur -> Pour intégrer la prise en compte d'un **ID** dynamique dans l'url
- Modification des routes -> Principalement celles de *web.php,* afin d'ajouter une route unique pour les matières, une route unique pour les thèmes, et une route unique pour les chapitres ainsi que la variable **{id}** à chacune de ces 3 routes
- Modification des modèles *class Matiere{}, Theme{}* et *Chapitre{}* : pour y ajouter les **requêtes** permettant de récupérer les bons IDs dynamiquement
- Modification de *IndexController,* bien sûr, pour **écraser les redondances** et les remplacer par des fonctions dynamiques

2. Tableau des modifications apportées au projet

N°	Fichier / Composant	Modification apportée	Justification
1	**Routeur**	Intégration d'un paramètre dynamique **{id}** dans les routes	Permettre l'affichage dynamique d'une ressource selon son identifiant
2	**Fichier de routes**	Remplacement des routes redondantes par des routes dynamiques /matiere/{id}, /theme/{id}, /chapitre/{id}	Réduire la duplication de code et centraliser la définition des routes
3	**Model Matiere**	Ajout des requêtes SQL nécessaires à la récupération dynamique des données par id	Alimenter les vues avec les données correspondant à la ressource demandée
3	**Model Theme**	Ajout des requêtes SQL nécessaires à la récupération dynamique des données par id	Idem
3	**Model Chapitre**	Ajout des requêtes SQL nécessaires à la récupération dynamique des données par id	Idem
4	**IndexController**	Remplacement des fonctions redondantes par 3 fonctions dynamiques : matiere($id), theme($id), chapitre($id)	Appliquer le principe DRY *(Don't Repeat Yourself)* et simplifier le contrôleur

Figure 4

3. Preuves des modifications et aperçu avant / après

A. Modifications du Routeur de routes

Extrait du fichier / Classe Router.php

Avant

```php
public function run()
{
    $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
    $method = $_SERVER['REQUEST_METHOD'];

    foreach (Route::getRoutes() as $route) {
        if ($route['method'] === $method && $route['path'] === $uri) {      CostantiniYH, il y
            foreach ($route['middleware'] as $middlewareName) {
                $middlewareClass = 'Middlewares\\' . ucfirst($middlewareName) . "Middleware";
                $middleware = new $middlewareClass();
                $middleware->handle();
            }

            // var_dump($uri);
            $this->callHandler($route['handler']);
            return;
        }
    }
    http_response_code(404);
    echo "404 - Page non trouvé";
}

1 reference
private function callHandler($handler)
{
    if (is_string($handler) && str_contains($handler, '@')) {
        list($controllerPath, $method) = explode('@', $handler);
        $controllerClass = 'App\\Controller\\' . str_replace('/', '\\', $controllerPath);
        $controllerObj = new $controllerClass($this->pdo);
        $controllerObj->$method([redacted]);
    }

    if (is_callable($handler)) {
        $handler();
    }
}
```

Figure 5

A ce stade, la fonction ***run()*** récupère les *routes* et éventuellement, les *middlewares* s'il y'en a, puis appelle la fonction ***callHandler()***. *callHandler()* analyse le *handler* de la route qui est composé d'un *Controller* et d'une méthode séparé par une arobase @, puis, elle instancie le *Controller* en indiquant la méthode à retourner.

Extrait de la classe Routeur avec modification et ajout du paramètre :

- Transformation de la *$route['path']* en groupe de capture regex.
- Ajout des délimiteurs de la regex.
- Isoler le *$matches*, correspondant à l'ID dans le tableau, avec *array_shift* qui va supprimer l'élément [0] du tableau.
- Nommer l'ID *$params*
- Et l'ajouter en paramètre des fonctions nécessaires au *Controller*.

Après

```php
public function run()
{
    $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
    $method = $_SERVER['REQUEST_METHOD'];

    foreach (Route::getRoutes() as $route) {
        if ($route['method'] !== $method) continue;

        // Convertit {id} en groupe de capture regex
        $pattern = preg_replace('/\{[^}]+\}/', '([^/]+)', $route['path']);
        $pattern = '#^' . $pattern . '$#';

        if (preg_match($pattern, $uri, $matches)) {
            array_shift($matches);
            $params = $matches;

            foreach ($route['middleware'] as $middlewareName) {
                $middlewareClass = 'Middlewares\\' . ucfirst($middlewareName) . "Middleware";
                $middleware = new $middlewareClass();
                $middleware->handle();
            }

            // var_dump($uri);
            $this->callHandler($route['handler'], $params);
            return;
        }                    You, maintenant • Uncommitted changes
    }
    http_response_code(404);
    echo "404 - Page non trouvé";
}

1 reference
private function callHandler($handler, $params = [])
{
    if (is_string($handler) && str_contains($handler, '@')) {
        list($controllerPath, $method) = explode('@', $handler);
        $controllerClass = 'App\\Controller\\' . str_replace('/', '\\', $controllerPath);
        $controllerObj = new $controllerClass($this->pdo);
        $controllerObj->$method(...$params);
    }

    if (is_callable($handler)) {
        $handler(...$params);
    }
}
```

Figure 6

B. Modifications des routes dans *web.php*

Passage de 8 routes à 3 routes.

Avant

```php
Route::get('/', 'IndexController@index');        You, le mois

Route::get('/support', 'IndexController@indexSupport');
Route::get('/sisr', 'IndexController@indexSisr');
Route::get('/slam', 'IndexController@indexSlam');
Route::get('/cybersecurite', 'IndexController@indexCyber');

Route::get('/mathematiques', 'IndexController@indexMath');
Route::get('/cejm', 'IndexController@indexCejm');
Route::get('/cge', 'IndexController@indexCge');
Route::get('/anglais', 'IndexController@indexAnglais');
```

Figure 8

Après

```php
Route::get('/', 'IndexController@index');
Route::get("/matieres", 'IndexController@matieres');
Route::get("/matiere/{id}", 'IndexController@matiere');
Route::get("/theme/{id}", 'IndexController@theme');
```

Figure 8

C. Modifications des Models (Ajout des requêtes)

Modification Matiere :

```
public function getMatiereById($id) {
    $stmt = $this->query("SELECT * FROM $this->table
    WHERE id = $id
    ORDER BY id");
    return $stmt->fetch(PDO::FETCH_ASSOC);
}
```

Figure 9

Ajout de la méthode *getMatiereById()* qui nous permettra de récupérer le nom (qui sera le titre) à partir de l'ID de la matière sélectionnée, transmis par l'URL. Voir par la suite, la méthode *matiere($id)* du contrôleur.

Modification de Theme

Ajout de la méthode *getThemeByMatiereId()* pour afficher les thèmes en fonction de l'ID stocké dans l'url.

```
public function getThemeByMatiereId($id) {
    $stmt = $this->query(
        "SELECT * FROM {$this->table}
        WHERE matieres_id = $id
        ORDER BY id");
    return $stmt->fetchAll();
}

1 reference | 0 overrides
public function getThemeById($id) {
    $stmt = $this->query(
        "SELECT * FROM {$this->table}
        WHERE id = $id
        ORDER BY id");
    return $stmt->fetch();
}
```

Figure 10

Modification de Chapitre

```
public function getChapitreByThemeId($id) {        You,
    $stmt = $this->query("SELECT * FROM {$this->table}
                          WHERE themes_id = $id
                          ORDER BY id");
    return $stmt->fetchAll();
}
```

Figure 11

Ajout de la méthode *getChapitreByThemeId()* pour afficher les chapitres en fonction du thème sélectionné.

D. Modification du *Controller* principal

```php
// Liste des matières
0 references | 0 overrides
public function matieres() {
    // Navbar + cartes
    $matieres = $this->Matieres->getMatiere();

    afficherVue('Accueil', "", "public/matieres", "", "", "", [
        'matieres' => $matieres
    ]);
}

// Liste des thèmes de la matière slectionnée
 0 references | 0 overrides
 public function matiere($id) {
    // Récupérer toutes les matières pour la navbar
    $listMatieres = $this->Matieres->getMatiere();
    // Sélectionner la matière correspondante à l'id stocké
    // dans l'url pour afficher son nom
    $matiere = $this->Matieres->getMatiereById($id);
    // Sélectionner tous les thèmes associés à l'id de la matière
    $themes = $this->Themes->getThemeByMatiereId($id);

    afficherVue($matiere['name'], "/matiere", 'public/themes',
     "", "", "", [
        'matieres' => $listMatieres,
        'themes' => $themes
    ]);
}
```

La méthode *matiere()* affiche la liste des matières avec *getMatiere()* qui renvoie toutes les lignes de la table *matières* sans condition.

La méthode *matiere(**$id**)* sert à afficher les thèmes en fonction de la matière sélectionnée. Mais aussi, à afficher le nom de la matière en titre.

Figure 12

```php
// Liste des chapitres du thème sélectionné
0 references | 0 overrides
public function theme($id) {
    // Récupérer toutes les matières pour la navbar ou autre
    $listMatieres = $this->Matieres->getMatiere();
    // Récupérer la ligne du theme à partir de l'id stocké dans l'url
    // pour le titre et le fil d'ariane
    $themes = $this->Themes->getThemeById($id);
    // Récupérer les lignes de la table chapitres selon le thème
    $chapitres = $this->Chapitres->getChapitreByThemeId($id);

    afficherVue($themes['name'], "", 'public/chapitres', "", "", "", [
        'matieres' => $listMatieres,
        'themes' => $themes,
        'chapitres' => $chapitres
    ]);
}
```

Enfin, la méthode *theme(**$id**)* permet d'afficher dynamiquement les chapitres en fonction du thème sélectionné avec *getChapitreByThemID($id).*

On utilise *getThemeById($id)* pour afficher le titre et utiliser l'ID dans le fil d'ariane pour revenir en arrière sur la matière en question.

Figure 13

IV. Amélioration de l'interface utilisateur

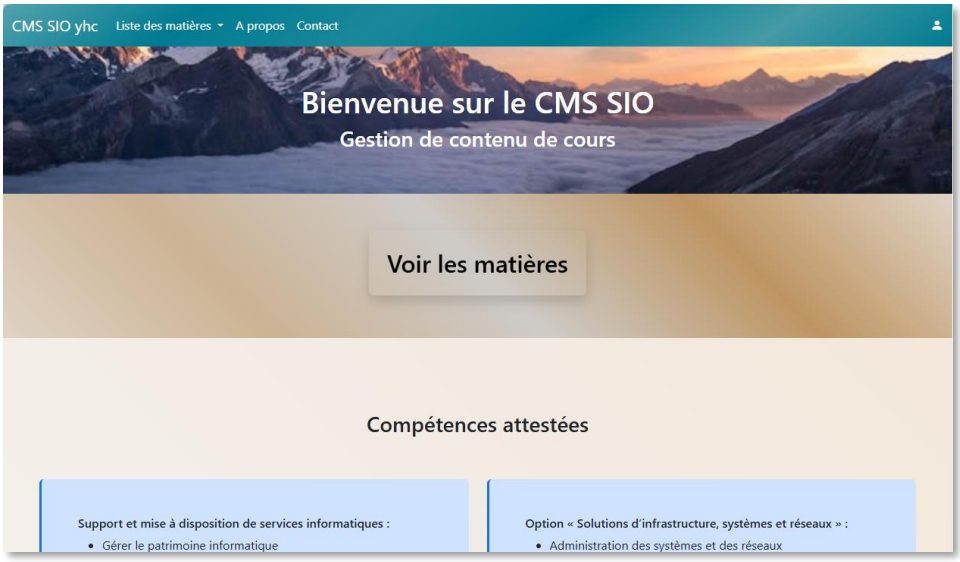

Figure 14

Comme prévu, j'ai libéré la page d'accueil afin de la rendre plus conviviale, et en y ajoutant un bouton pour accéder aux matières.

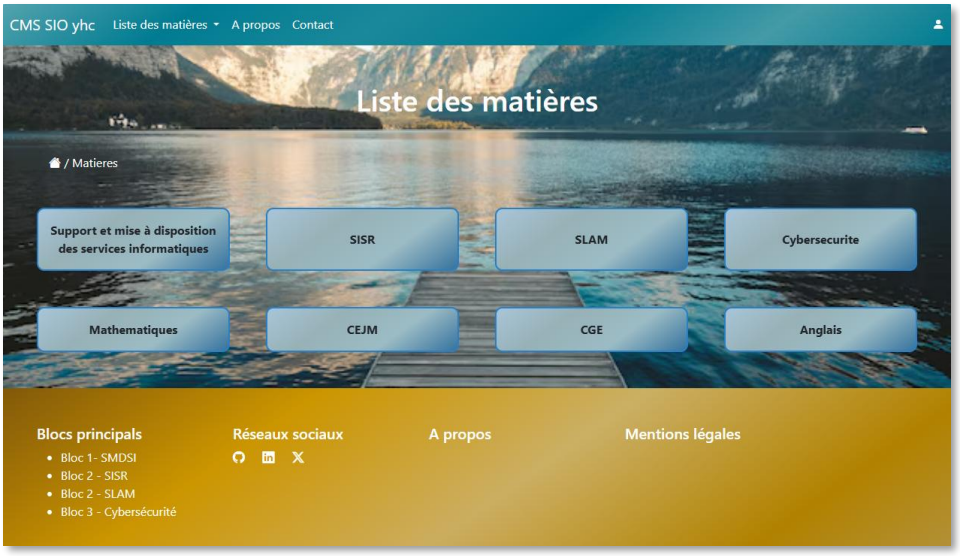

Figure 15

La liste des matières a donc été déplacée vers une nouvelle page dédiée.

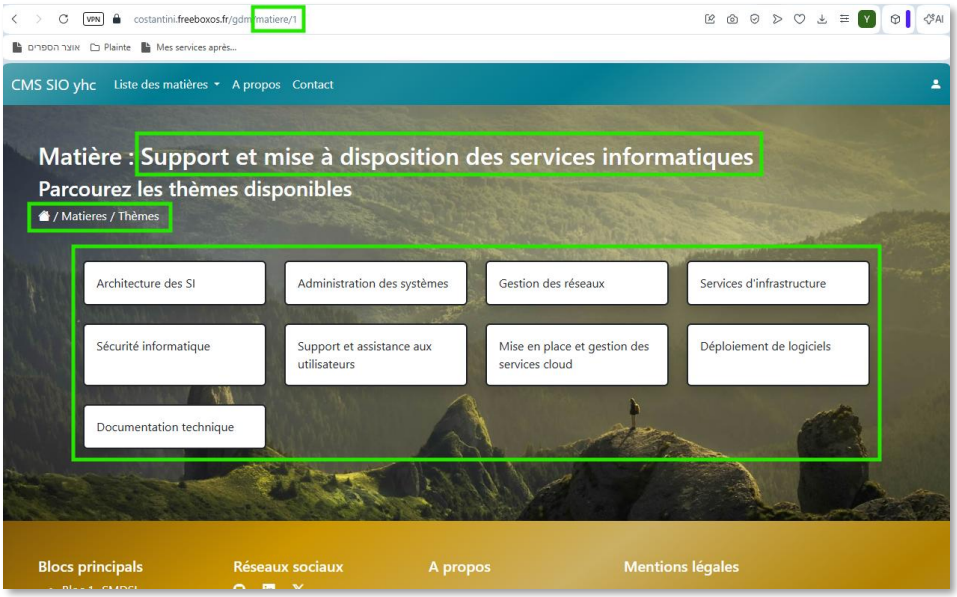

Figure 16

Voici la page des thèmes après sélection d'une matière :

- On aperçoit dans l'URL que la matière 1 a été sélectionnée
- On voit bien son titre : « Support et mise à disposition des services informatiques »
- Les thèmes associés à la matière sélectionnée sont visibles et correspondent bien à la base de données

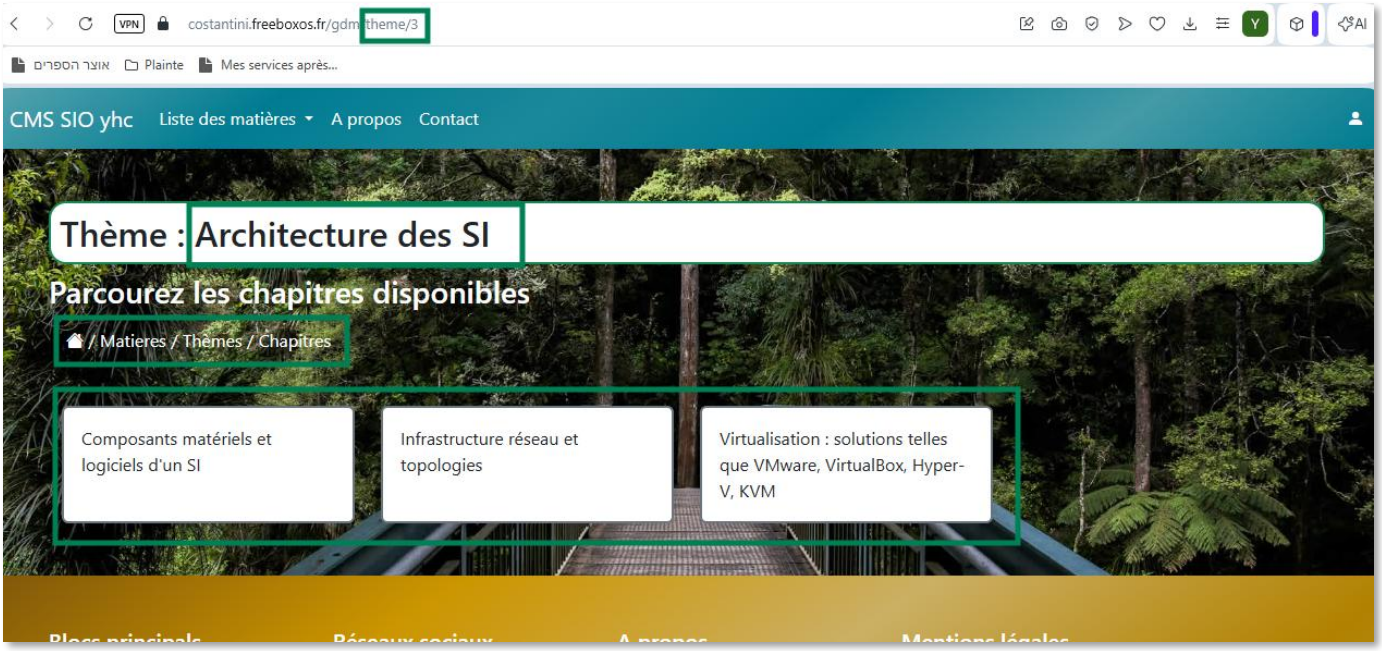

Figure 17

Enfin, l'interface des chapitres. De la même manière que pour les thèmes, on peut apercevoir :

- Le thème sélectionné en haut dans l'URL et son titre « Architecture des SI » qui correspond au thème de la matière précédemment sélectionnée.
- Le fil d'ariane permettant de revenir exactement sur la matière du thème sélectionné.
- Et les chapitres affichés proprement et nettement, correspondants au thèmes sélectionné.

CONCLUSION
Cette activité m'a permis de mettre en pratique le principe fondamental DRY dans un contexte de maintenance évolutive réelle. La modification la plus technique a été l'adaptation du routeur pour qu'il convertisse les paramètres {id} en groupes de capture regex, permettant ensuite de transmettre dynamiquement l'identifiant au contrôleur. Un point d'amélioration identifié concerne les requêtes des modèles *Theme* et *Chapitre* : les méthodes *getThemeByMatiereId()* et *getChapitreByThemeId()* auraient pu intégrer une jointure SQL afin de récupérer en une seule requête à la fois les données de la ressource et celles de son parent, évitant ainsi deux appels séparés à la base de données. Cette optimisation constitue une piste d'amélioration concrète pour une prochaine itération. L'utilisation de Git tout au long du processus a permis de travailler sereinement en conservant un historique des modifications.

EVOLUTION POSSIBLE
➢ Implémenter les jointures SQL dans *getThemeByMatiereId*() et *getChapitreByThemeId*() pour réduire le nombre de requêtes à la base de données ➢ Ajouter un fil d'Ariane complet et fonctionnel sur toutes les pages pour améliorer la navigation ➢ Mettre en place une gestion des erreurs 404 personnalisée lorsqu'un id inexistant est transmis dans l'URL ➢ Introduire un système de cache pour les requêtes fréquentes afin d'optimiser les performances